coding_doc.pdf

Code generation record for Homework Assignment 1

1. Chatbot(s) used

This draft records the use of ChatGPT only. No other chatbot was used in the current code-generation session.

2. Prompt log

Prompt 1 (exact transcript used in this session):

```
What exactly is being asked and what do I need to do from this assignment given to me? I want to
have the code written and any other files that would needed for this assignment to be completed
to the fullest and have no errors. Please follow the guidelines within the file and please
provide anything that will be beneficial to completing this assignment successfully.
```

Summary of what this prompt generated: the complete movie_recommender.py program with a CLI, file-loading functions, ranking functions, genre-preference logic, recommendation logic, docstrings, and Python 3.12-compatible code; the complete test_movie_recommender.py tester that creates temporary input files, loads them, calls each required function, prints results, and compares them against expected values; and two PDF documentation drafts describing the coding and testing process.

No separate prompt was used for the tester. The same prompt requested the code, the test file, and any other files needed for the assignment.

3. Case sensitivity decisions

- Movie names are preserved exactly as they appear in the movies file for display, but matched internally in a case-insensitive way using normalized text. This prevents failures if the ratings file writes the same movie title with different capitalization.
- Genre names are also matched case-insensitively so that inputs such as Action, action, and ACTION are treated as the same genre when filtering and computing user preference.
- User IDs are compared case-insensitively in the current implementation. This avoids accidental duplicate or missed matches caused by capitalization differences such as u1 versus U1.
- The code does not alter the displayed movie title or displayed genre label; it only normalizes internally for matching and validation.
- Ratings are kept as raw numeric values and are never rounded during computation, matching the assignment requirement.

4. Parts of the generated code that may require explanation

No separate follow-up explanation prompt was used in this run before the final code was assembled. However, the following parts are the ones most likely to need explanation during review:

Code element	Why it may need explanation	How it was made easier to follow
defaultdict	Used to accumulate totals and counts without repetitive key checks.	Short helper logic and clear variable names were used.
normalize_text(...).casefold()	Used for case-insensitive matching that is more robust than lower().	The normalization behavior is isolated in one helper function.
sorted(..., key=lambda ...)	Used to rank movies and genres by score, with alphabetical tie-breaking.	Sorting rules are centralized in helper functions and documented with docstrings.

Code element	Why it may need explanation	How it was made easier to follow
Temporary files in the tester	Used so the tester can load input files without requiring permanent data files in the final zip.	The tester writes data, runs checks, then deletes everything automatically.

5. Defensive measures built into the generated code

- Strips whitespace around every field before processing.
- Skips blank lines.
- Rejects rows with the wrong number of pipe-delimited fields.
- Rejects rows with missing required values.
- Rejects ratings that are not numeric.
- Rejects ratings outside the inclusive 0-5 range.
- Rejects ratings for movies that do not appear in the movies file.
- Rejects duplicate movie titles in the movies file to avoid ambiguous lookups.
- Rejects duplicate user/movie ratings to enforce the assignment rule that a user can rate a particular movie only once.
- Collects warnings for skipped rows so the CLI can expose data-quality issues during testing.

6. Additional issues considered

Potential issues not explicitly handled in the final code include embedded pipe characters inside movie titles, conflicting duplicate movie records that use different IDs for the same displayed title, and non-UTF-8 text files. These were not added because the assignment format strongly suggests clean pipe-delimited text files. If the dataset were expanded in the future, those cases would justify a follow-up prompt requesting stricter file validation and encoding error handling.

7. Final coding summary

The final program satisfies the listed coding requirements: it is an executable Python program named movie_recommender.py, provides a CLI menu, implements the required recommendation and ranking features, documents each function with docstrings, and is compatible with Python 3.12.